If already registered, click to check your payment status

Course outline

About NPTEL
How does an NPTEL online course work?
Week 1
Week 2
Week 3
week 4
Week 5
Week 6
Week 7
Week 8
Week 9
- Natural Language Processing - Author Stylometry
- Natural Language Processing - Author Stylometry - Part 01
- Natural Language Processing - Author Stylometry - Part 02
- Natural Language Processing - Author Stylometry - Part 03
- Natural Language Processing - Author Stylometry - Part 04
- Natural Language Processing - Author Stylometry - Part 05
- Natural Language Processing - Author Stylometry - Part 06
- Natural Language Processing - Author Stylometry - Part 07
- Natural Language Processing - Author Stylometry - Part 08
- Natural Language Processing - Author Stylometry - Part 09
- Natural Language Processing - Author Stylometry - Part 10
- Introduction to Networkx - Part 01
- Introduction to Networkx - Part 02
- Six Degrees of Separation : Meet your favourites
- Six Degrees of Separation : Meet your favourites - Part 01
- Six Degrees of Separation : Meet your favourites - Part 02
- Six Degrees of Separation : Meet your favourites - Part 03
- Area Calculation - Don't Measure
- Area Calculation - Don't Measure - Part 01
- Area Calculation - Don't Measure - Part 02
- Area Calculation - Don't Measure - Part 03
- Area Calculation - Don't Measure - Part 04
- Area Calculation - Don't Measure - Part 05
- Area Calculation - Don't Measure - Part 06
- Week 9 Feedback Form: The Joy of Computing using Python
- Quiz: Week 9 : Assignment 9
- Week 9 : Programming Assignment 1
- Week 9 : Programming Assignment 2
- Week 9 : Programming Assignment 3
- Week 9 Solution

Week 10
Week 11
Week 12
Text Transcripts
Download Videos
Problem Solving Session - July 2025

Week 9 : Assignment 9

The due date for submitting this assignment has passed.

Due on 2025-09-24, 23:59 IST.

Assignment submitted on 2025-09-23, 06:46 IST

Many real-world systems can be understood better when modeled as networks. Think about social networks like Instagram, road maps connecting cities, biological pathways, or the structure of the internet. Each of these can be represented using nodes (the entities) and edges (the relationships between them).

To work with such networks in Python, we use a library called NetworkX. This library helps us easily build and analyze networks using just a few lines of code.

Let's walk through a basic example and understand what each line does:

```
import networkx as nx
import matplotlib.pyplot as plt
```

We begin by importing the necessary libraries. networkx is used for creating and analyzing the graph, while matplotlib.pyplot helps us visualize it.

```
G = nx.Graph()
```

This line creates an empty undirected graph named G. Think of this as starting with a blank canvas where we'll add people (nodes) and their connections (edges).

```
G.add_node("Alice")
G.add_node("Bob")
```

We add two individual nodes to our graph: "Alice" and "Bob". A node can represent any entity : a person, a city, or even a webpage.

```
G.add_edge("Alice", "Bob")
```

This adds a connection between Alice and Bob. If either of these nodes didn't exist already, NetworkX would automatically add them first.

```
G.add_edges_from([("Bob", "Charlie"), ("Alice", "David")])
```

Here, we add multiple edges at once. This line connects Bob to Charlie and Alice to David. Again, if "Charlie" or "David" weren't added earlier, they will be automatically created.

```
nx.draw(G, with_labels=True)
plt.show()
```

This code is used for drawing the graph. nx.draw() creates a visual layout, and with_labels=True ensures that node names (like "Alice") are shown. plt.show() then displays the graph window.

```
print("Nodes:", G.nodes())
print("Edges:", G.edges())
print("Number of nodes:", G.number_of_nodes())
print("Number of edges:", G.number_of_edges())
```

These lines print basic information about the graph.

- G.nodes() gives a list of all nodes.
- G.edges() shows all the connections.
- G.number_of_nodes() counts how many unique entities are in the graph.
- G.number_of_edges() counts how many relationships exist between them.

This simple example shows how easy it is to model and explore networks using NetworkX. From here, you can move on to more complex tasks like finding the shortest path, checking connected components, or analyzing social structures.

1) What will be the output of the following code? *1 point*

```
import networkx as nx
G = nx.Graph()
G.add_edge("A", "B")
print("C" in G.nodes())
```

○ True
● False
○ Error: "C" is not defined
○ None

Yes, the answer is correct.
Score: 1
Accepted Answers:
False

2) Predict the output of the following code. *1 point*

```
import networkx as nx
G = nx.Graph()
G.add_edges_from([("X", "Y"), ("Y", "Z")])
print(G.number_of_nodes())
```

○ 2
● 3
○ 1
○ Error

Yes, the answer is correct.
Score: 1
Accepted Answers:
3

3) What does G.add_edges_from([("A", "B"), ("B", "C")]) do? *1 point*

○ Adds nodes only if they already exist
● Adds edges and automatically creates any missing nodes
○ Only creates the nodes without connecting them
○ Adds weighted edges between existing nodes

Yes, the answer is correct.
Score: 1
Accepted Answers:
Adds edges and automatically creates any missing nodes

4) What is the output of the given code ? *1 point*

```
import networkx as nx
G = nx.Graph()
G.add_node("Tom")
print(G.number_of_edges())
```

○ 1
● 0
○ Error
○ Depends on the number of nodes

Yes, the answer is correct.
Score: 1
Accepted Answers:
0

5) Which of the following is TRUE after executing only this line? *1 point*

```
G = nx.Graph()
```

○ One empty node is created
● A graph with no nodes or edges is created
○ A directed graph is created
○ A graph with one edge and no node is created

Yes, the answer is correct.
Score: 1
Accepted Answers:
A graph with no nodes or edges is created

6) Why is plt.show() used after nx.draw(G, with_labels=True)? *1 point*

○ To print the node names
● To display the graph visualization
○ To calculate shortest path
○ To show node degrees

Yes, the answer is correct.
Score: 1
Accepted Answers:
To display the graph visualization

7) What happens if you add an edge like G.add_edge("A", "B") when "A" and "B" are not yet added? *1 point*

○ Error: You must add nodes first
○ Only the edge is created, not the nodes
● Nodes "A" and "B" are automatically added
○ Only "A" is added, "B" is ignored

Yes, the answer is correct.
Score: 1
Accepted Answers:
Nodes "A" and "B" are automatically added

8) Predict the output *1 point*

```
import networkx as nx
G = nx.Graph()
G.add_node("A")
G.add_node("B")
print(G.edges())
```

○ [('A', 'B')]
● [ ]
○ ['A', 'B']
○ Error

Yes, the answer is correct.
Score: 1
Accepted Answers:
[ ]

9) You are building a graph step-by-step using the following sequence of operations: *1 point*

1. Create an empty graph using G = nx.Graph()
2. Add edge: G.add_edge("A", "B")
3. Add edge: G.add_edge("B", "C")
4. Add node: G.add_node("D")

How many nodes and edges will exist in the graph after these steps?

● 4 nodes, 2 edges
○ 3 nodes, 2 edges
○ 3 nodes, 3 edges
○ 4 nodes, 3 edges

Yes, the answer is correct.
Score: 1
Accepted Answers:
4 nodes, 2 edges

10) Analyse the code given below : *1 point*

```
G = nx.Graph()
G.add_nodes_from(["A", "B", "C", "D"])
G.add_edges_from([("A", "B"), ("B", "C"), ("A", "C")])
nx.draw(G, with_labels=True)
plt.show()
```

Which of the following statements is most accurate?

○ Node D will not be visible in the graph because it has no edges
● Node D will be visible but disconnected
○ Node D is automatically removed before drawing
○ The graph will throw an error because node D is isolated

Yes, the answer is correct.
Score: 1
Accepted Answers:
Node D will be visible but disconnected

Once you've created a graph using NetworkX by adding nodes and edges, the next step is to analyze its structure. NetworkX provides several built-in functions to help us understand how different parts of the network are connected.

Let's take this example:

```
import networkx as nx
G = nx.Graph()

G.add_edges_from([("Alice", "Bob"), ("Bob", "Charlie"), ("Charlie", "David"), ("Alice", "David")])
```

Degree of a Node
The degree of a node is the number of edges connected to it. For example:

```
print(G.degree("Bob"))
```

This will return 2 because Bob is connected to both Alice and Charlie.

Neighbors of a Node
To get all nodes connected to a particular node (its neighbors), we can use:

```
list(G.neighbors("Charlie"))
```

This will return ['Bob', 'David'] because Charlie is connected to both.

Checking for Paths
To check if there is a path between two nodes, we can use:

```
nx.has_path(G, "Alice", "Charlie")
```

This returns True because Alice → Bob → Charlie is a valid path.

Shortest Path
To find the shortest connection between two nodes:

```
nx.shortest_path(G, "Alice", "Charlie")
```

These tools are powerful in understanding how information flows through a network, how connected the network is, and how close different nodes are to each other.

11) Given the following graph construction: *1 point*

```
G = nx.Graph()
G.add_edges_from([("Alice", "Bob"), ("Bob", "Charlie"), ("Charlie", "David"), ("Alice", "David")])
```

What will be the output of G.degree("Charlie")?

○ 1
● 2
○ 3
○ 4

Yes, the answer is correct.
Score: 1
Accepted Answers:
2

12) What is the correct way to find the neighbors of "Bob"? *1 point*

○ G.neighbors("Bob")
● list(G.neighbors("Bob"))
○ G.degree("Bob")
○ nx.neighbors(G, "Bob")

Yes, the answer is correct.
Score: 1
Accepted Answers:
list(G.neighbors("Bob"))

13) *1 point*

```
G = nx.Graph()
G.add_edges_from([("Alice", "Bob"), ("Bob", "Charlie"), ("Charlie", "David"), ("Alice", "David")])
```

For this graph, what does nx.shortest_path(G, "Alice", "Charlie") return?

○ ['Alice', 'Charlie']
● ['Alice', 'David', 'Charlie']
○ ['Alice', 'Bob', 'Charlie']
○ Both B and C are valid paths

No, the answer is incorrect.
Score: 0
Accepted Answers:
['Alice', 'Bob', 'Charlie']

14) What will be the number of nodes after executing the following code? *1 point*

```
edges = [("A", "B"), ("B", "C"), ("C", "D")]
G = nx.Graph()
G.add_edges_from(edges)
print(len(G.nodes()))
```

○ 2
○ 3
● 4
○ Cannot be determined

Yes, the answer is correct.
Score: 1
Accepted Answers:
4

15) Consider the following code: *1 point*

```
G = nx.Graph()
G.add_edges_from([("A", "B"), ("B", "C")])
degrees = {node: G.degree(node) for node in G.nodes()}
```

What will be the value of degrees["B"]?

○ 1
● 2
○ 3
○ Error

Yes, the answer is correct.
Score: 1
Accepted Answers:
2

16) Which of the following will correctly collect the neighbors of all nodes in a dictionary? *1 point*

○ {n: G.neighbors(n) for n in G.nodes()}
● {n: list(G.neighbors(n)) for n in G.nodes()}
○ {n: G.degree(n) for n in G.nodes()}
○ {n: nx.shortest_path(n) for n in G.nodes()}

Yes, the answer is correct.
Score: 1
Accepted Answers:
{n: list(G.neighbors(n)) for n in G.nodes()}

17) Given the graph: *1 point*

```
G = nx.Graph()
G.add_edges_from([("A", "B"), ("B", "C")])
path = nx.shortest_path(G, "A", "C")
```

What is the value of path[1]?

○ "A"
● "B"
○ "C"
○ Error

Yes, the answer is correct.
Score: 1
Accepted Answers:
"B"

18) What will be the output of the following code? *1 point*

```
G = nx.Graph()
G.add_edges_from([("A", "B"), ("C", "D")])
pairs = [("A", "B"), ("A", "C")]
results = [nx.has_path(G, u, v) for u, v in pairs]
```

What is the value of results?

○ [True, True]
● [False, False]
○ [True, False]
○ Error

No, the answer is incorrect.
Score: 0
Accepted Answers:
[True, False]

19) What does the following code store in paths? *1 point*

```
G = nx.Graph()
G.add_edges_from([("X", "Y"), ("Y", "Z")])
nodes = ["X", "Z"]
paths = {n: nx.shortest_path(G, "X", n) for n in nodes}
```

● Paths from "X" to all other nodes
○ Error, cannot use dict with shortest_path
○ Shortest path from "X" to "X" and "X" to "Z"
○ Only the path from "X" to "Z"

No, the answer is incorrect.
Score: 0
Accepted Answers:
Shortest path from "X" to "X" and "X" to "Z"

20) Given: *1 point*

```
G = nx.Graph()
G.add_edges_from([("A", "B"), ("B", "C"), ("C", "D")])
high_deg = [n for n in G.nodes() if G.degree(n) > 1]
```

What is high_deg?

● ["B", "C"]
○ ["A", "D"]
○ ["A", "B", "C"]
○ [ ]

Yes, the answer is correct.
Score: 1
Accepted Answers:
["B", "C"]

[Deleted question group]